Зміст

1 Умовний оператор if 2

1.1 Оператори порівнювання 2

1.2 Синтаксичні форми умовного оператора 3

2 Логічні оператори (and, or, not) 7

3 Генерація випадкових чисел: модуль random 11

Умовний оператор if

1

За замовчуванням хід виконання програми є лінійним: інструкції виконуються послідовно — рядок за рядком, від першого до останнього. Послідовне виконання є базою, але реальні алгоритми вимагають гнучкості та адаптивності.

У більшості задач програма повинна приймати рішення залежно від вхідних даних чи поточного стану обчислень. Для цього використовуються розгалуження (або умовні переходи).

Оператор if в Python — це

умовний оператор, який дозволяє програмі приймати рішення та виконувати певний блок коду лише тоді, коли виконується задана умова

В основі прийняття рішень у програмі лежать оператори порівняння (відношення). Вони дозволяють зіставляти значення виразів і повертають результатом булеве значення — **True (істина)** або **False (хибність)**.

1.1 Оператори порівнювання

Python підтримує базові математичні операції відношення:

```
Дорівнює:                a == b
Не дорівнює:             a != b
Менше ніж:               a < b
Менше або дорівнює:     a <= b
Більше ніж:              a > b
Більше або дорівнює:    a >= b
```

Зверніть увагу:

Оператор перевірки на **рівність** позначено двома знаками ==, оскільки один знак = в Python зарезервовано для операції **присвоєння**.

На відміну від багатьох інших мов програмування, Python дозволяє використовувати ланцюгові порівняння (подвійні нерівності), що звично для математиків, наприклад: **a < x <= b** замість запису через логічне «і».

Отримані булеві вирази є фундаментом для керування потоком виконання про-

грами: вони використовуються в умовних конструкціях (if, elif, else) та циклах (while).

1.2 Синтаксичні форми умовного оператора

Залежно від кількості напрямків (гілок) виконання програми, в Python застосовують три основні форми розгалуження:

1. Неповна форма (коротке розгалуження)

Використовується, коли певну дію потрібно виконати **тільки** за умови, що вираз-умова є істинним (`True`). Якщо умова хибна (`False`), блок коду просто пропускається.

```
Неповна форма
if вираз_умова1:
    оператор1
    оператор2
```

2. Повна форма (двохгілкове розгалуження)

Застосовується, коли є два альтернативних шляхи: один виконується при істинності умови (`True`), а інший (блок `else`) — якщо умова хибна (`False`).

```
Повна форма
if вираз_умова1:
    оператор1
    оператор2
else:
    Блок_операторів_3
```

3. Багатогілкова форма (багатоальтернативне розгалуження)

Використовується для перевірки серії послідовних умов за допомогою конструкцій `elif` (скорочення від *else if*). Умови перевіряються зверху вниз: виконується блок першої умови, яка виявиться істинною, після чого весь оператор `if` завершує роботу.

```
Багатогілкова форма
if вираз_умова1:
    Блок_операторів_1
elif вираз_умова2:
    Блок_операторів_2
elif вираз_умова_n:
    Блок_операторів_n
else:
   Блок_операторів_n+1
```

Важливі синтаксичні особливості Python:

☑ **Двокрапка (:):** є обов'язковою в кінці кожного рядка з `if`, `elif` та `else`. Вона позначає початок відповідного блоку коду.

Приклад 1. Користувач вводить число та отримує відповідь яке воно позитивне негативне або нуль.

```
x=int(input("Введіть x "))
if x>0:
    print("x позитивне")
else:
    print("x негативне або нуль")
    input()
```

Приклад 2. Користувач вводить два числа та отримує відповідь яке більше або дорівнюють.

```
A=input()
B=input()
if A>B: print("A більше") #можно писати у тому ж рядку один оп-р
elif A==B: print("A дорівнює B")
else: print("A менше B")
```

Приклад 3. Користувач отримує відповідь чи оплачений чек (ціна записана у коді, або можна ввести з клавіатури). (звернути увагу на імена змінних

```
print ("Привіт")
price1 = 50 #price1=int(input())
price2 = 32 #price2=int(input())
if price1+ price2 > 99 :
    print ("Сума не достатня")
else:
    print ("Чек оплачений")
    print ("До побачення")
```

Приклад 4. Вивести на екран місяці відповідної попи року

```
s=input("Введіть пору року")
if s=="зима":
    print("грудень")
    print("січень")
    print("лютий")
elif s=="весна":
    print("березень")
    print("квітень")
    print("травень")
elif s=="літо":
    print("червень")
    print("липень")
    print("серпень")
else:
    print("вересень")
    print("жовтень ")
    print("листопад")
print ("До побачення")
```

Контрольні питання до глави 4

1. Що таке умовний оператор `if` у Python? Для чого він використовується?
2. Які оператори порівняння підтримує Python? Наведіть приклади.
3. Чим відрізняється оператор присвоєння = від оператора перевірки на рівність ==?
4. Що таке ланцюгові порівняння у Python? Наведіть приклад.
5. Які основні синтаксичні форми умовного оператора існують у Python?

6. У чому полягає відмінність між неповною та повною формою умовного оператора?
7. Як працює багатогілкова форма умовного оператора з `elif`?
8. Яка роль двокрапки `:` у синтаксисі умовних операторів?
9. Що відбувається, якщо умова у операторі `if` є хибною?
10. Як можна записати умовний оператор у одному рядку?

Практичні завдання до глави 4

1. Реалізуйте програму, яка за двома введеними числами перевіряє, чи можна знайти частку від ділення першого на друге.
2. Напишіть програму, яка за введеними координатами точки вознaчає чверть.
3. Складіть програму, яка визначає найбільше з трьох введених чисел.
4. Реалізуйте програму, яка перевіряє, чи введене число є парним або непарним.
5. Напишіть програму, яка визначає, чи введене число кратне 10.
6. Складіть програму, яка за введеним віком користувача виводить повідомлення: «дитина», «підліток», «дорослий», «пенсіонер».
7. Реалізуйте програму, яка перевіряє, чи введений рік є високосним.
8. Напишіть програму, яка за введеною оцінкою (від 1 до 12) виводить текстове пояснення («незадовільно», «задовільно», «добре», «відмінно»).

Логічні оператори (and, or, not)

2

Прості умовні вирази (операції відношення) можна об'єднувати у складні **складені умови** за допомогою **логічних операторів** (булевих зв'язок). В Python використовуються три базові логічні оператори: **`and`, `or` та `not`**.

1. Кон'юнкція (`and` — логічне «І»)

Оператор **`and`** повертає **`True`** тоді й лише тоді, коли **обидва** операнди є істинними. Якщо хоча б один із виразів хибний (**`False`**), увесь складений вираз набуває значення **`False`**.

Пріоритет оператора **`and`** нижчий за пріоритет операторів порівняння (>, <, == тощо), тому у виразах на зразок:

Перевірка діапазону

```
if length > 8 and length < 20:
    print("Довжина відповідає вимогам")
```

дужки навколо окремих порівнянь писати не обов'язково.

2. Диз'юнкція (`or` — логічне «АБО»)

Оператор **`or`** повертає **`True`**, якщо **хоча б один** із операндів є істинним (або обидва одночасно). Вираз набуває значення **`False`** лише тоді, коли всі складові частини є хибними.

Приклад диз'юнкції

```
if age < 18 or age >= 60:
    print("Пільгова категорія")
```

3. Заперечення (`not` — логічне «НЕ»)

Унарний оператор **`not`** інвертує (змінює на протилежне) значення булевого виразу: **`True`** перетворюється на **`False`**, а **`False`** — на **`True`**.

Приклад заперечення

```
if not is_valid:
    print("Помилка введення даних")
```

Таблиця 2.1: Таблиця істинності для логічних операцій

Значення операндів		Результат операції		
X	Y	not X	X and Y	X or Y
False	False	True	False	False
False	True	True	False	True
True	False	False	False	True
True	True	False	True	True

Особливості Python:

- **Пріоритет виконання:** Найвищий пріоритет має `not`, далі йде `and`, і найнижчий — `or`. Для зміни порядку обчислення використовують круглі дужки `()`.
- **Ліниві обчислення (Short-circuit evaluation):** Python обчислює логічні вирази зліва направо і зупиняється, як тільки результат стає однозначним (наприклад, якщо у виразі з `and` перший операнд `False`, другий операнд навіть не обчислюється).

Приклад 1. Перевірка діапазону числа

```
x = int(input("Введіть число: "))
if x >= 10 and x <= 20:
    print("Число входить у діапазон від 10 до 20")
else:
    print("Число поза діапазоном")
```

Приклад 2. Перевірка категорії за віком

```
a = int(input("Введіть сторону a: "))
b = int(input("Введіть сторону b: "))
c = int(input("Введіть сторону c: "))

if a == b or a == c or b == c:
    print("Трикутник є рівнобедреним")
else:
    print("Трикутник не є рівнобедреним")
```

Приклад 3. Використання заперечення

```
password = input("Введіть пароль: ")
if not password:
    print("Пароль не може бути порожнім!")
else:
    print("Пароль прийнято")
```

Перевірка: чи число належить до множини парних чисел від 10 до 20

```
x = int(input("Введіть число: "))
if (x % 2 == 0) and (x >= 10 and x <= 20):
    print("Число належить до множини парних чисел у діапазоні [10;20]"
```

Контрольні питання до глави 5

1. Які логічні оператори підтримує Python? Для чого вони використовуються?
2. Що таке кон'юнкція (`and`)? Коли вираз з `and` повертає істину?
3. Що таке диз'юнкція (`or`)? У яких випадках вираз з `or` є істинним?
4. Як працює оператор заперечення (`not`)?
5. Який порядок пріоритетів виконання логічних операторів у Python?
6. Що таке «ліниві обчислення» (short-circuit evaluation)? Наведіть приклад.
7. Як можна комбінувати логічні оператори з операторами порівняння?
8. Наведіть приклад використання `and` для перевірки належності числа до діапазону.
9. Наведіть приклад використання `or` для перевірки кількох альтернативних умов.
10. Як можна використати `not` для перевірки, що число не входить у певний інтервал?
11. Що таке таблиця істинності? Як вона допомагає зрозуміти роботу логічних операторів?
12. Чому дужки важливі при комбінуванні кількох логічних операторів у складних виразах?

Практичні завдання до глави 5

1. Напишіть програму, яка перевіряє, чи введене число належить до діапазону від 100 до 200 включно.

2. Напишіть програму, яка перевіряє, чи трикутник є рівнобедреним (дві сторони рівні), а не рівностороннім.
3. Складіть програму, яка перевіряє, чи число є парним і належить до діапазону від 50 до 100.
4. Реалізуйте програму, яка перевіряє, чи введене число є кратним 3 або 7.
5. Складіть програму, яка визначає, чи введене число є парним або кратним 5.
6. Реалізуйте програму, яка перевіряє, чи введене число належить до множини парних чисел у діапазоні [10;20].

Генерація випадкових чисел: модуль random

3

При розв'язанні багатьох математичних задач, моделюванні ймовірнісних процесів (метод Монте-Карло) або створенні ігор виникає потреба у використанні **випадкових чисел**. У Python для цього призначений вбудований модуль `random`.

Щоб отримати доступ до функцій модуля, його необхідно імпортувати на початку програми:

Імпорт модуля

```
import random
```

Основні функції модуля random

1. Дійсні числа: `random.random()` та `random.uniform()`

- ☑ `random.random()` — повертає випадкове дійсне число x у напіввідкритому інтервалі $[0.0, 1.0)$.
- ☑ `random.uniform(a, b)` — повертає випадкове дійсне число x у відрізку $[a, b]$.

Генерація дійсних чисел

```
import random
# Випадкове число від 0.0 до 1.0
p = random.random()
print("Імовірність",p)

# Випадкова температура в межах [-5.0, 15.0]
temp = random.uniform(-5.0, 15.0)
print("Температура:", temp)
```

2. Цілі числа: `random.randint()` та `random.randrange()`

- ☑ `random.randint(a, b)` — повертає випадкове ціле число $n \in [a, b]$ (включаючи обидва кінці a та b).

- ☑ `random.randrange(start, stop, step)` — повертає випадковий елемент із множини цылих чисел із діапазону від start до stop-1 включно з кроком step.

Генерація цілих чисел

```
import random
# Моделювання кидка грального кубика (від 1 до 6)
dice = random.randint(1, 6)
print("Випало число:", dice)

# Випадкове парне число від 10 до 98
even_num = random.randrange(10, 100, 2)
print("Парне число:",even_num)
```

Відтворюваність результатів: `random.seed()`

У Python модуль random генерує не справжні випадкові числа, а псевдовипадкові. Це означає, що вони створюються за допомогою математичного алгоритму (наприклад, Вихор Мерсенна), який формує послідовність чисел, що виглядають випадковими, але насправді повністю визначаються початковим значенням — seed.

Якщо не задавати seed, генератор використовує системний час, і послідовність буде різною при кожному запуску програми.

Якщо задати random.seed(42), то кожен запуск програми відтворюватиме одну й ту саму послідовність чисел. Це важливо для:

наукових експериментів — щоб результати можна було перевірити й повторити;

тестування програм — щоб відтворювати однакові сценарії;

навчання — щоб приклади завжди давали однаковий результат.

Оскільки Python генерує **псевдовипадкові числа** за допомогою алгоритму (Вихр Мерсенна), послідовність обчислень залежить від початкового значення (значення ініціалізації — *seed*).

Функція `random.seed(a)` фіксує початковий стан генератора. Це є критично важливим у наукових та математичних обчисленнях для **відтворюваності результатів експерименту**.

Фіксація генератора

```
import random

random.seed(42)  # Задаємо фіксоване зерно
print(random.randint(1, 100))  # Завжди повертатиме один і той самий р
```

Контрольні питання до глави 7

1. Для чого у Python використовується модуль `random`?
2. Які основні функції модуля `random` призначені для генерації дійсних чисел?
3. У чому різниця між `random.random()` та `random.uniform(a, b)`?
4. Які функції використовуються для генерації цілих чисел? Наведіть приклади.
5. Як працює функція `random.randint(a, b)`?
6. Як працює функція `random.randrange(start, stop, step)`?
7. Наведіть приклад використання генератора випадкових чисел для моделювання кидка кубика.
8. Як можна згенерувати випадкове парне число у заданому діапазоні?
9. Що означає поняття «псевдовипадкові числа»?
10. Яку роль відіграє функція `random.seed()`?
11. Чому важливо задавати `seed` у наукових експериментах та тестуванні програм?
12. Який алгоритм використовується у Python для генерації псевдовипадкових чисел?
13. Що відбувається, якщо не задавати `seed`? Як формується послідовність чисел?
14. Як можна забезпечити відтворюваність результатів при використанні модуля `random`?